



Victor Manuel Touza Rodal

Business Intelligence Portfolio

Python • PostgreSQL • Power BI

About Me

Industrial Engineer graduated from the University of Vigo, specialized in Mechanical Engineering, with more than twenty years of experience in the automotive industry as Mechanical Design Engineer, Information Manager and Project Coordinator.

Throughout my professional career, I have focused on understanding complex technical processes, structuring information and supporting decision-making through a rigorous and analytical approach.

This experience has naturally led me to Data Analytics and Business Intelligence, where the goal remains the same: transforming data into meaningful information that supports better decisions.



In April 2026, I completed a Master's Degree in Big Data and Business Intelligence at the Universidad Europea Miguel de Cervantes. During the programme, I developed a complete Business Intelligence project, applying Python (pandas, NumPy), SQL (PostgreSQL) and Power BI to acquire, transform and analyse data, design a relational database and create interactive dashboards for decision support.

Today, my objective is to combine my industrial background with modern data analytics techniques to contribute to process optimisation and data-driven decision-making as a Data Analyst or Business Intelligence Analyst.

Technical Skills

- **Programming:** Python (pandas, NumPy)
- **Databases:** SQL (PostgreSQL)
- **Business Intelligence:** Power BI
- **Engineering:** Mechanical Design, Process Analysis, Project Coordination

Business Intelligence for the Medical Device Market

I. Context

Advancements in technology, scientific research, and artificial intelligence have significantly accelerated production and reduced costs for medical equipment and devices. Consequently, the healthcare sector has become a highly attractive market for business. However, selecting which products to manufacture and identifying the right target markets requires a thorough analysis of each country's specific situation.

II. Objective

The objective of this project is to determine which countries and medical devices offer the highest potential for successful international commercialization.

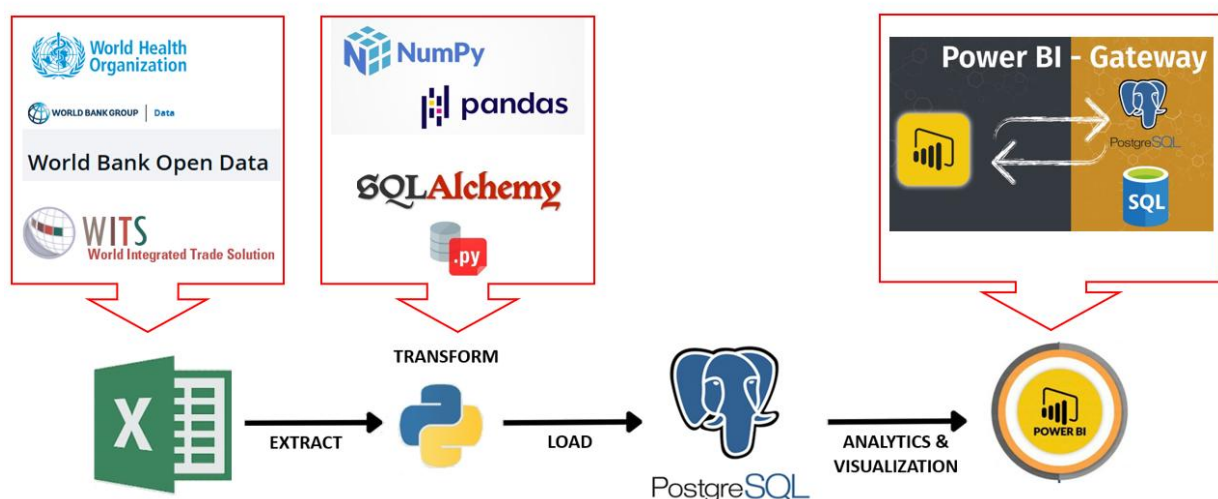


Fig. 1: Technical description of the analysis process.

Project Information

Project Attribute	Technical Specifications
Project Type	Master's Final Project
Duration	6 months
Methodology	ETL + Business Intelligence (BI)
Tools & Technologies	Python, PostgreSQL, Power BI
Data Sources	WHO, World Bank, WITS

III. System Architecture

The project was designed as a complete Business Intelligence workflow, covering the entire process from data acquisition to decision support.

Data from different international sources were collected, transformed and integrated through an ETL process. The resulting dataset was analysed to calculate key performance indicators (KPIs), stored in a PostgreSQL database and finally presented through interactive Power BI dashboards to support business decisions.

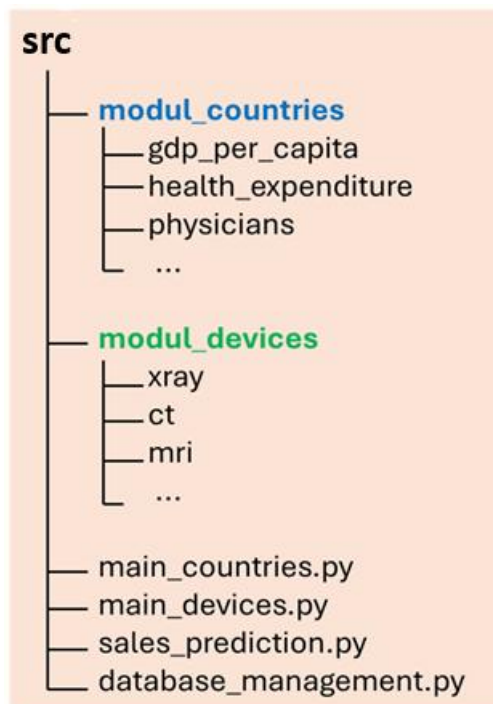


Fig. 2: Architecture of the analysis process.

The resulting analytical data are presented through interactive Power BI dashboards to support data-driven decision-making.

IV. Python

The project was developed using a modular architecture.

Each data source was processed independently before being integrated into a common analytical workflow. This approach simplified maintenance, testing and future extensions of the project.

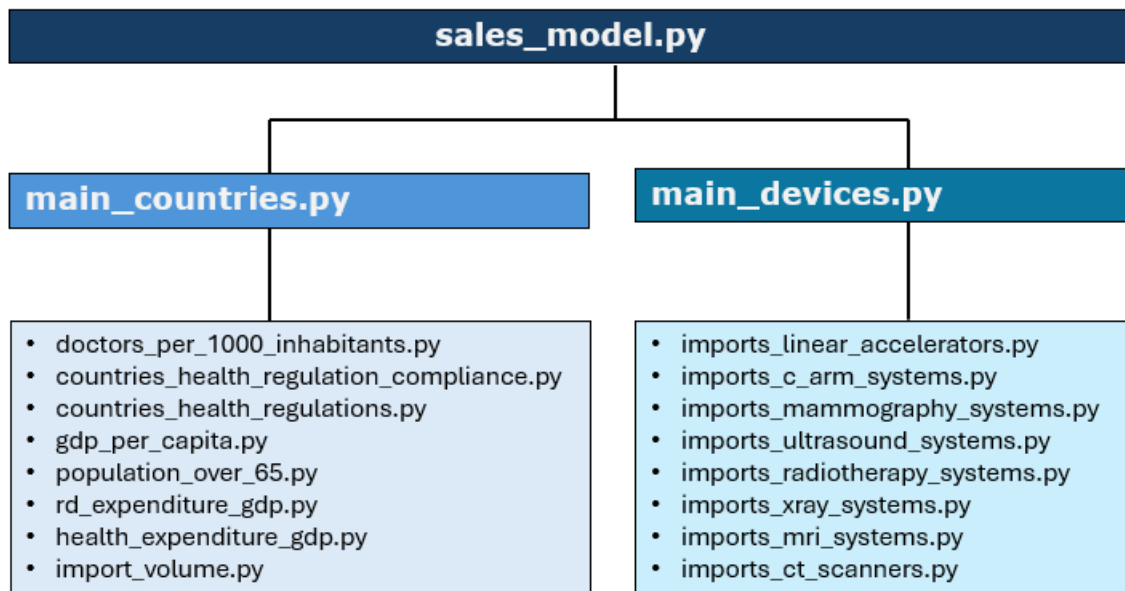


Fig. 3: Modular structure.

1. Individual modules

The following section presents an excerpt from the **imports_ultrasound_systems.py** module, located at the lowest level of the project's modular architecture. This module is responsible for importing, cleaning, and standardizing the raw data before it is integrated into the analytical pipeline.

```
def load_ultrasound_systems():  
  
    file_path = "C:/TFM/Datasets/Product_Selection/3_imports_ultrasound_systems"  
  
    files = [  
        f for f in os.listdir(file_path)  
        if f.startswith("901812_Ultrasound_Systems_")  
        and f.endswith(".xlsx")  
    ]  
  
    list_df = []
```

```

for file in files:
    year = file.split("_")[-1].split(".")[0]
    df = pd.read_excel(os.path.join(file_path, file))
    df["Year"] = int(year)
    list_df.append(df)

df_ultrasound_systems = pd.concat(
    list_df,
    ignore_index=True
)

df_ultrasound_systems = (
    df_ultrasound_systems
    .dropna(axis=1, how="all")
    .drop(columns=columns_to_remove, errors="ignore")
    .rename(columns={
        "Reporter": "Country",
        "Commodity Code": "Product_Code",
        "Trade Value (US$)": "Trade Value 1000USD"
    })
)

df_ultrasound_systems["Product_Name"] = "Ultrasound Systems"

df_ultrasound_systems["Trade Value 1000USD"] = (
    df_ultrasound_systems["Trade Value 1000USD"]
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
    .astype(float) / 1000
)

df_ultrasound_systems["Country_Code"] = (
    df_ultrasound_systems["Country"]
    .apply(get_country_code)
)

return df_ultrasound_systems

```

2. Modular Architecture: Device Integration

Objective

The second layer of the modular architecture integrates all device-specific analytical modules into a single dataset. It automatically imports the processed datasets, calculates product attractiveness indicators and generates a unified dataset ready for the final sales analysis.

Dynamic module integration

Each medical device is processed independently in its own module. The parent module dynamically imports and executes every child module.

```
child_modules = {
    "c_arms_systems": "modul_devices.imports_c_arm_systems.period_2010_2024",
    "ultrasound_systems":
        "modul_devices.imports_ultrasound_systems.period_2010_2024",
    "xray_systems": "modul_devices.imports_xray_systems.period_2010_2024",
    "linear_accelerators":
        "modul_devices.imports_linear_accelerators.period_2010_2024",
    "mammography_systems":
        "modul_devices.imports_mammography_systems.period_2010_2024",
    "radiotherapy_systems":
        "modul_devices.imports_radiotherapy_systems.period_2010_2024",
    "mri_systems": "modul_devices.imports_mri_systems.period_2010_2024",
    "ct_scanners": "modul_devices.imports_ct_scanners.period_2010_2024"
}

df_ultrasound_systems = import_module(
    child_modules["ultrasound_systems"]
).load_ultrasound_systems()
```

Key concepts

- Modular architecture
- Dynamic imports
- Reusable analytical modules
- Scalable project design

Dataset integration

Once every module has been executed, all datasets are combined into a single DataFrame.

```
dataframes = [
    df_c_arms_systems,
    df_ultrasound_systems,
    df_xray_systems,
    df_linear_accelerators,
    df_mammography_systems,
    df_radiotherapy_systems,
    df_mri_systems,
    df_ct_scanners
]

df_devices = (
    pd.concat(dataframes, ignore_index=True)
```

```

        .sort_values("Country_Code")
        .reset_index(drop=True)
    )

```

Applied techniques

- Data integration
- `pd.concat()`
- DataFrame sorting
- Index management

Dynamic weighting by year

Recent imports receive greater importance than older observations through a dynamic weighting coefficient.

```

current_year = pd.Timestamp.today().year

df_devices["Coefficient_Year"] = (
    1 - 0.02 * (current_year - df_devices["Year"])
).clip(lower=0)

```

This coefficient introduces a time-dependent weighting into the analytical model, giving higher relevance to more recent market behaviour.

Product normalization

Trade values are normalized independently for every country-product combination.

```

df_devices["Standardized_Trade"] = (
    df_devices
    .groupby(["Country", "Product_Code"])["Trade Value 1000USD"]
    .transform(lambda x: x / x.max() if x.max() != 0 else 0)
)

df_devices["Annual_Product_Index"] = (
    df_devices["Standardized_Trade"]
    * df_devices["Coefficient_Year"]
)

```

Applied techniques

- GroupBy
- Transform
- Feature engineering

- Custom KPI calculation

Global Product Index

Finally, annual product indices are aggregated to obtain a Global Product Index for every Country–Product combination.

```
df_sum_product = (
    df_devices
    .groupby(
        ["Country",
        "Country_Code",
        "Product_Name",
        "Product_Code"]
    )
    .agg(
        Global_Product_Index=(
            "Annual_Product_Index",
            "sum"
        )
    )
    .reset_index()
)

df_product_index = df_devices.merge(
    df_sum_product,
    on=[
        "Country",
        "Country_Code",
        "Product_Name",
        "Product_Code"
    ],
    how="left"
)
```

Applied techniques

- Aggregation
- GroupBy
- Merge
- KPI generation

3. Final Integration: Sales Analysis

Objective

The final module integrates both analytical branches of the project:

- Country attractiveness (AMI)
- Product attractiveness (Global Product Index)

It combines both indicators into a unified Sales Index that supports strategic decision-making by identifying the most attractive medical devices and target markets.

Integration of analytical modules

```
from main_countries import run_main_countries
from main_devices import run_main_devices

df_countries = run_main_countries()
df_devices = run_main_devices()

df_analysis = pd.merge(
    df_devices,
    df_countries,
    on="Country_Code",
    how="inner"
)
```

Applied techniques

- Modular architecture
- Parent-child integration
- Data merging
- Relational data modelling

Construction of the Sales Index

```
weight_country = 0.6
weight_device = 0.4

df_analysis["Annual_Sales_Index"] = (
    weight_country * df_analysis["AMI"] +
    weight_device * df_analysis["Annual_Product_Index"]
)

df_analysis["Global_Sales_Index"] = (
    weight_country * df_analysis["AMI"] +
    weight_device * df_analysis["Global_Product_Index"]
)
```

Business logic

The model combines two complementary dimensions:

- Market attractiveness (country)
- Product attractiveness (medical device)

The resulting Sales Index provides an objective ranking of country-product opportunities.

Statistical validation

```
statistics = (
    df_analysis[statistics_columns]
    .describe()
    .T
)

statistics["median"] = (
    df_analysis[statistics_columns]
    .median()
)

statistics["standard deviation"] = (
    df_analysis[statistics_columns]
    .std()
)
```

The project automatically generates descriptive statistics together with outlier detection using the Interquartile Range (IQR) method.

Final outputs

The analytical pipeline automatically produces:

- Clean analytical dataset
- Annual Product Index
- Global Product Index
- Annual Sales Index
- Global Sales Index
- Descriptive statistics
- Excel and CSV reports
- Power BI ready dataset

Optional Forecasting Module

The project also includes an experimental forecasting module based on Holt's Exponential Smoothing. This module predicts the Annual Sales Index for each Country–Product combination using historical observations from 2010-2024.

Techniques

- Time Series Forecasting
- Holt Exponential Smoothing
- MAE
- RMSE

4. Database Management with PostgreSQL

This module is responsible for integrating the analytical pipeline with a PostgreSQL relational database. Its purpose is to automatically create the database schema and export the results generated by the analytical modules.

The program performs four main tasks:

- Creates the database tables if they do not already exist.
- Populates the dimension tables (*countries* and *products*).
- Stores the historical analytical results.
- Stores the predictive model outputs, including both sales forecasts and performance metrics.

The implementation uses **SQLAlchemy** to establish the database connection and execute SQL statements, allowing Python to interact directly with PostgreSQL while preserving referential integrity and preventing duplicate records through the use of **ON CONFLICT** clauses.

Creating the database schema

The first stage automatically creates the relational schema composed of five tables: **countries**, **products**, **results**, **sales_forecast**, and **predictive_model_metrics**.

This approach makes the application self-contained, as the database structure is generated programmatically without requiring manual SQL scripts.

```
# =====
# 2. Create tables if they do not exist
# =====

def create_tables():
    with engine.begin() as conn:

        conn.execute(text("""
            CREATE TABLE IF NOT EXISTS countries (
                id_country SERIAL PRIMARY KEY,
                country_code VARCHAR(100) UNIQUE,
                country VARCHAR(100)
            );
        """))
```

```

conn.execute(text("""
    CREATE TABLE IF NOT EXISTS products (
        id_product SERIAL PRIMARY KEY,
        product_code VARCHAR(150) UNIQUE,
        product_name VARCHAR(150)
    );
"""))

```

Exporting analytical results

Once the analytical model has generated the final dataset, the program retrieves the corresponding primary keys from the dimension tables and inserts the historical results into PostgreSQL.

To ensure consistency, the implementation uses dictionaries to map country and product identifiers and updates existing records whenever a country–product–year combination already exists.

```

def insert_results(df_analysis_py):

    with engine.connect() as conn:
        df_countries_sql = pd.read_sql("SELECT id_country, country_code FROM countries",
conn)
        df_products_sql = pd.read_sql("SELECT id_product, product_name FROM products",
conn)

        dict_countries = dict(zip(df_countries_sql["country_code"],
df_countries_sql["id_country"]))
        dict_products = dict(zip(df_products_sql["product_name"],
df_products_sql["id_product"]))

        with engine.begin() as conn:
            for _, row in df_analysis_py.iterrows():

                id_country = dict_countries.get(row["Country_Code"])
                id_product = dict_products.get(row["Product_Name"])

                if id_country is None or id_product is None:
                    continue

                conn.execute(text("""
                    INSERT INTO results (
                        id_country, id_product, country, country_code,
                        product_name, product_code, year,

```

```

        annual_sales_index, global_sales_index
    )
VALUES (
    :id_country, :id_product, :country, :country_code,
    :product_name, :product_code, :year,
    :annual_index, :global_index
)
ON CONFLICT (id_country, id_product, year)
DO UPDATE SET
    annual_sales_index = EXCLUDED.annual_sales_index,
    global_sales_index = EXCLUDED.global_sales_index
"""), {
    "id_country": id_country,
    "id_product": id_product,
    "country": row["Country"],
    "country_code": row["Country_Code"],
    "product_name": row["Product_Name"],
    "product_code": row["Product_Code"],
    "year": int(row["Year"]),
    "annual_index": float(row["Annual_Sales_Index"]),
    "global_index": float(row["Global_Sales_Index"])
})

```

Exporting predictive outputs

The final stage stores the outputs generated by the forecasting model. Besides the predicted Sales Index values for future years, the program also exports the model evaluation metrics (MAE and RMSE), allowing the predictive performance to be analysed directly from the database.

```

def insert_sales_forecast(df_forecast_py):

    with engine.connect() as conn:
        df_countries_sql = pd.read_sql("SELECT id_country, country FROM countries", conn)
        df_products_sql = pd.read_sql("SELECT id_product, product_name FROM products",
conn)

        dict_countries = dict(zip(df_countries_sql["country"], df_countries_sql["id_country"]))
        dict_products = dict(zip(df_products_sql["product_name"],
df_products_sql["id_product"]))

        with engine.begin() as conn:
            for _, row in df_forecast_py.iterrows():

                id_country = dict_countries.get(row["Country"])
                id_product = dict_products.get(row["Product_Name"])

                if id_country is None or id_product is None:
                    continue

```

```

conn.execute(text("""
    INSERT INTO sales_forecast (
        id_country, id_product, year, predicted_sales_index
    )
    VALUES (:id_country, :id_product, :year,
        :predicted_sales_index)
    ON CONFLICT (id_country, id_product, year)
    DO UPDATE SET predicted_sales_index =
        EXCLUDED.predicted_sales_index
"""), {
    "id_country": id_country,
    "id_product": id_product,
    "year": int(row["Year"]),
    "predicted_sales_index": float(row["Predicted_Sales_Index"])
})

```

V. Database: Relational Database Design

The analytical results generated in Python are stored in a PostgreSQL relational database. The database follows a normalized structure composed of two dimension tables (*countries* and *products*) and three fact tables containing historical results, sales forecasts and predictive model performance metrics.

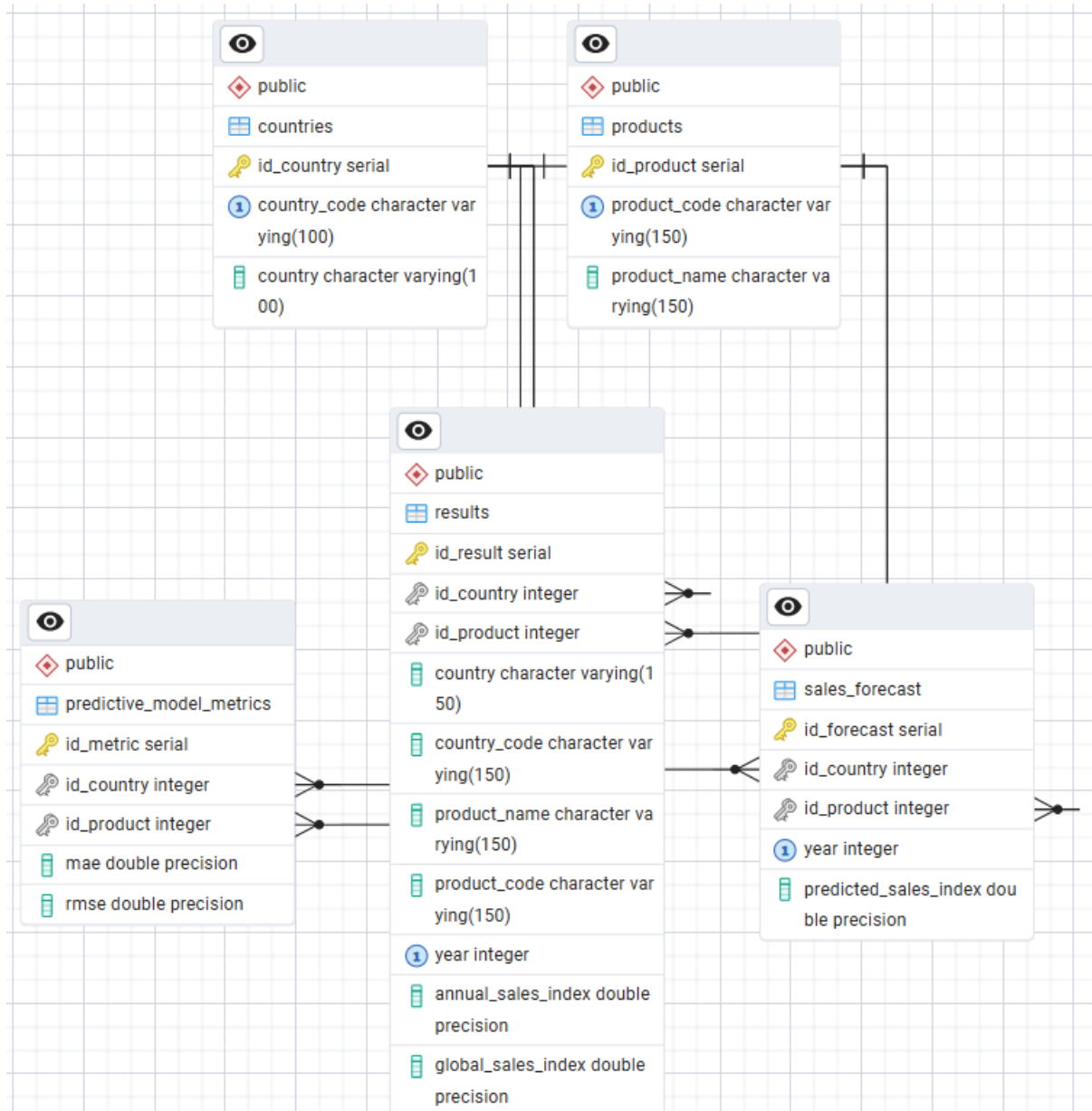


Fig. 4: Relational Database Design.

Database Schema

The PostgreSQL database was designed following a relational model composed of five tables.

The **countries** and **products** tables act as dimension tables, while **results**, **sales_forecast**, and **predictive_model_metrics** store the analytical outputs generated by the Python pipeline.

The relationships between the tables are implemented through foreign keys:

- **countries** → **results**
- **products** → **results**
- **countries** → **sales_forecast**
- **products** → **sales_forecast**
- **countries** → **predictive_model_metrics**
- **products** → **predictive_model_metrics**

This relational structure guarantees data integrity, avoids redundancy, and enables efficient querying from Power BI.

Table	Description
countries	Country master data
products	Healthcare product master data
results	Historical Sales Index values
sales_forecast	Forecasted Sales Index (2025–2027)
predictive_model_metrics	Holt model performance (MAE and RMSE)

VI. Dashboard

Power BI data model

The PostgreSQL database is imported into Power BI using a star-schema approach. The **Results** table acts as the central fact table, while **Countries** and **Products** provide the corresponding dimensions. Additional tables contain the forecasted Sales Index values and the predictive model performance metrics.

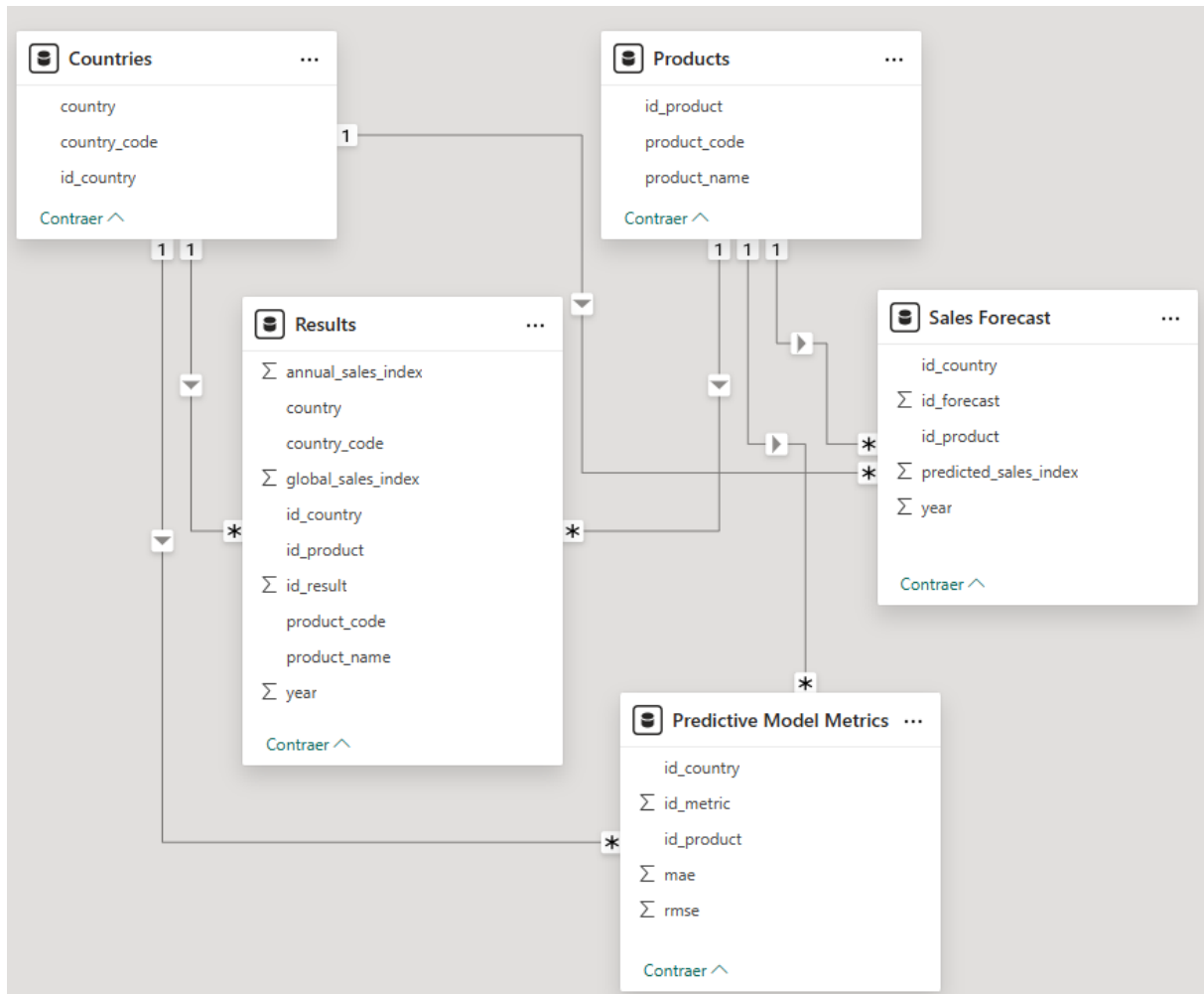


Fig. 5: PowerBI Data Model.

DAX Measures

To support the interactive dashboard, several DAX measures were created to aggregate the information stored in PostgreSQL and provide dynamic business indicators.

The measures can be grouped into three categories:

- **Aggregation measures**, used to calculate the Annual Sales Index and the Global Sales Index.

- **Ranking measures**, used to identify the country and the healthcare product with the highest commercial attractiveness under the current report filters.
- **Context-aware measures**, which calculate the Global Sales Index by country or by product while preserving the appropriate filter context.

These measures allow the dashboard to respond dynamically to user selections and provide meaningful business insights for decision-making.

Measure	Purpose
Average Sales Index	Calculates the average Annual Sales Index.
Total Annual Sales Index	Sums the Annual Sales Index across the selected data.
Total Global Sales Index	Sums the Global Sales Index across the selected data.
Top Country	Identifies the country with the highest Global Sales Index.
Top Product	Identifies the product with the highest Global Sales Index.
Total Global Sales Index by Country	Calculates the Global Sales Index grouped by country.
Total Global Sales Index by Product	Calculates the Global Sales Index grouped by product.

```

Top Country =
VAR TopCountry =
    TOPN(
        1,
        ALLSELECTED(Countries[country]),
        [Total Global Sales Index],
        DESC
    )
RETURN
CONCATENATEX(
    TopCountry,
    Countries[country],
    ", "
)

```

```

Total Global Sales Index by
Country =
CALCULATE(
    [Total Global Sales Index],
    ALLEXCEPT(
        Results,
        Results[country]
    )
)

```

Choropleth Map

Displays the geographical distribution of the Annual Sales Index across all analyzed countries. Darker colors indicate markets with higher commercial attractiveness.

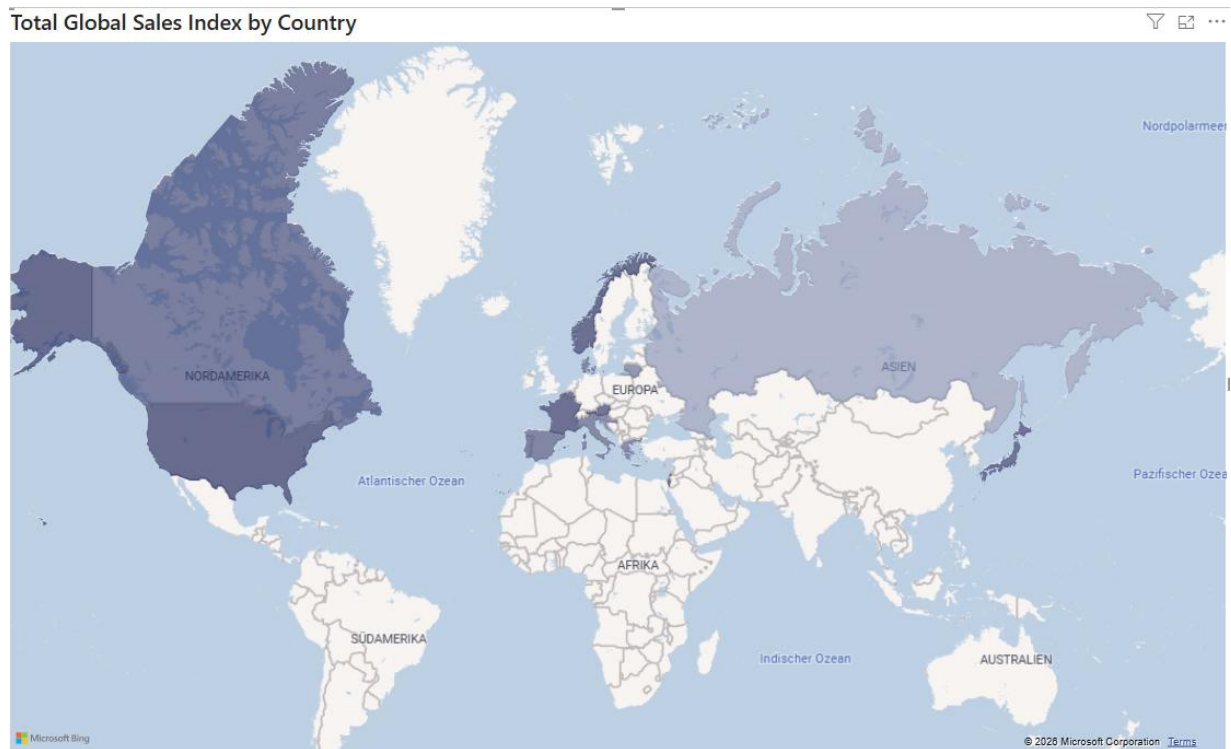


Fig. 6: Choropleth Map.

Country Ranking

Ranks countries according to their Total Annual Sales Index, allowing the identification of the most attractive healthcare markets.

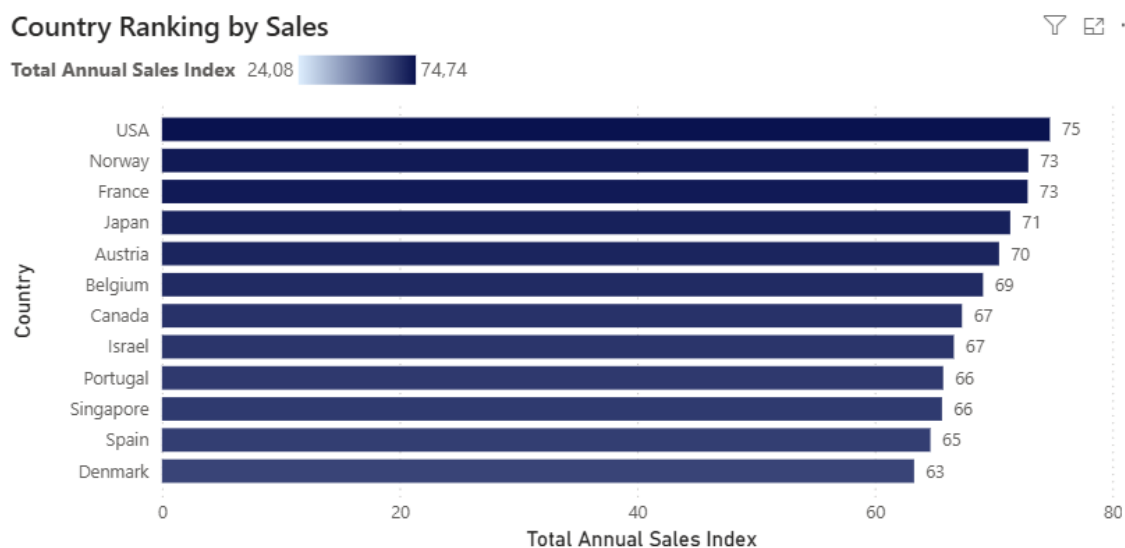


Fig. 7: Country Ranking.

Country Evolution

Shows the evolution of the Annual Sales Index over time for each country, enabling the comparison of market trends.

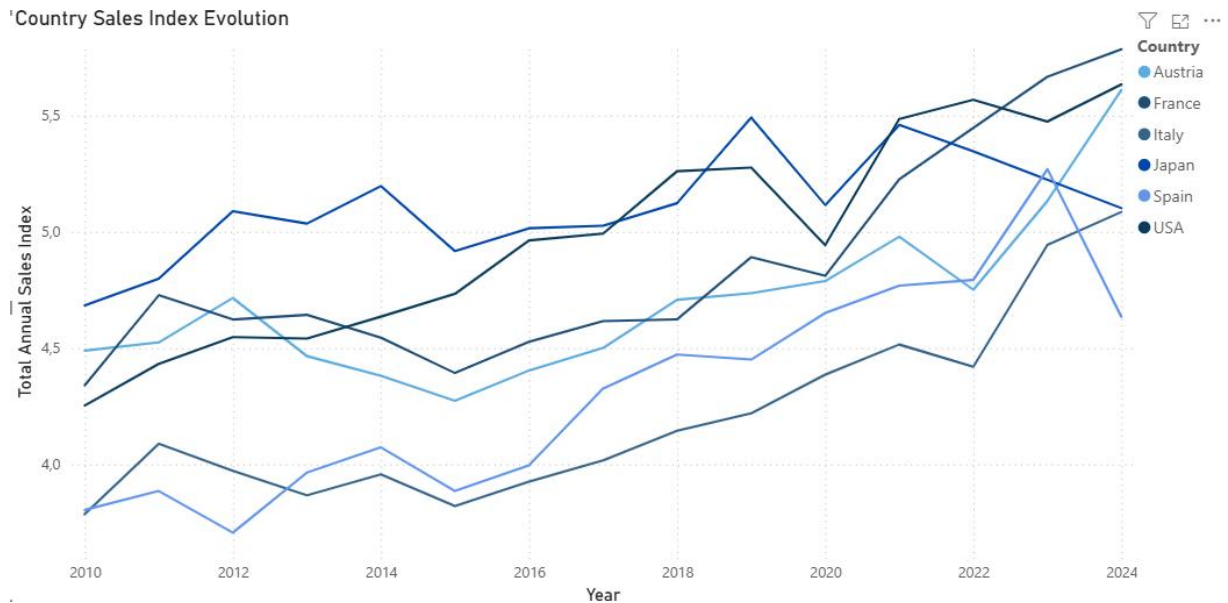


Fig. 8: Country Evolution.

Product Ranking

Compares the overall sales attractiveness of the different healthcare products included in the analysis.

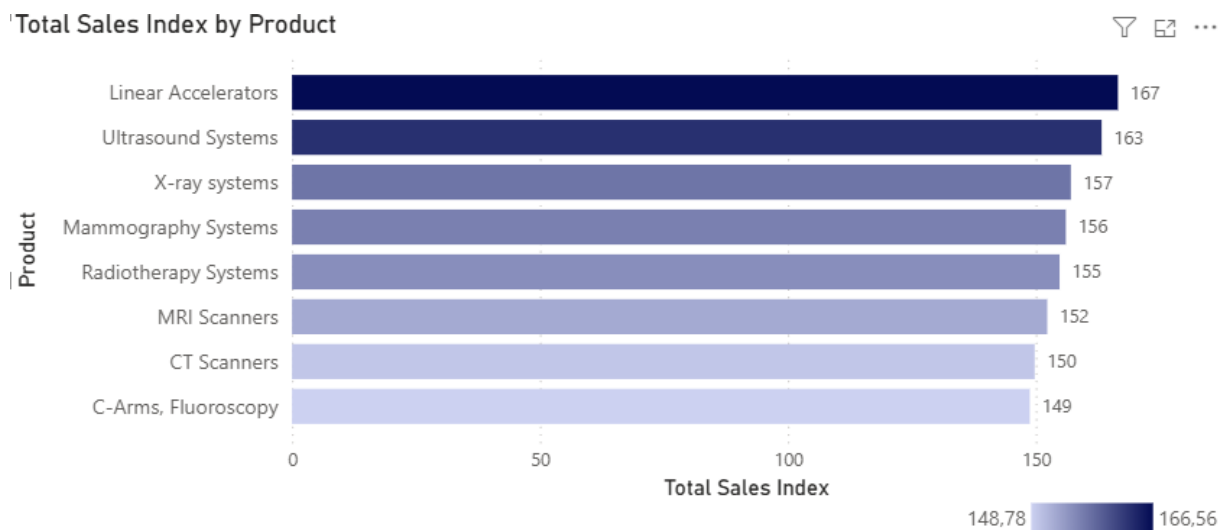


Fig. 9: Product Ranking.

Country × Product Matrix

Cross-analysis between countries and products to identify the combinations with the highest commercial potential.

country	C-Arms, Fluoroscopy	CT Scanners	Linear Accelerators	Mammography Systems	MRI Scanners	Radiotherapy Systems	Ultrasound Systems	X-ray systems	Total
Austria	8,19	8,80	9,22	8,66	8,87	8,99	9,43	8,30	70,46
Belgium	8,60	8,66	9,72	8,23	8,54	8,43	8,33	8,61	69,11
Canada	8,07	8,39	8,79	8,79	8,31	8,04	8,65	8,29	67,34
Croatia	6,24	6,59	8,49	7,81	6,39	6,61	7,76	7,42	57,30
Cuba	3,21	2,59	3,68	2,45	2,44	2,91	3,77	3,03	24,08
Denmark	7,44	7,76	8,26	6,96	8,15	8,33	8,73	7,67	63,31
France	9,26	8,43	9,45	9,68	8,58	9,06	9,79	8,62	72,87
Greece	7,17	7,09	8,22	7,73	7,68	8,18	8,52	7,48	62,08
Israel	7,92	7,81	8,35	8,52	8,24	8,63	8,92	8,27	66,66
Italy	6,28	8,13	9,37	7,15	8,28	7,79	7,46	8,69	63,15
Japan	7,96	9,05	9,34	8,60	9,23	8,64	9,13	9,45	71,40
Lithuania	6,98	7,26	6,89	7,47	6,90	6,60	8,23	7,24	57,57
Luxembourg	7,60	5,27	7,43	7,54	5,76	7,86	7,58	6,74	55,78
Norway	8,73	8,47	9,37	9,30	9,37	9,51	9,21	8,96	72,92
Portugal	7,30	8,25	9,44	8,69	7,77	8,31	8,43	7,58	65,76
Russia	5,49	5,84	6,02	6,24	5,81	5,35	5,89	6,24	46,88
Singapore	7,50	8,60	8,69	7,75	7,52	8,18	8,48	8,93	65,65
Slovenia	7,54	7,00	7,24	6,98	6,94	6,59	7,61	6,95	56,86
Spain	8,03	7,00	8,75	8,27	8,13	7,27	8,00	9,24	64,68
USA	9,27	8,77	9,82	9,25	9,41	9,51	9,36	9,35	74,74
Total	148,78	149,77	166,56	156,05	152,30	154,79	163,27	157,07	1.248,59

Fig. 10: Country × Product Matrix.

Stacked Column Chart

Displays the contribution of each product to the Global Sales Index within every country.

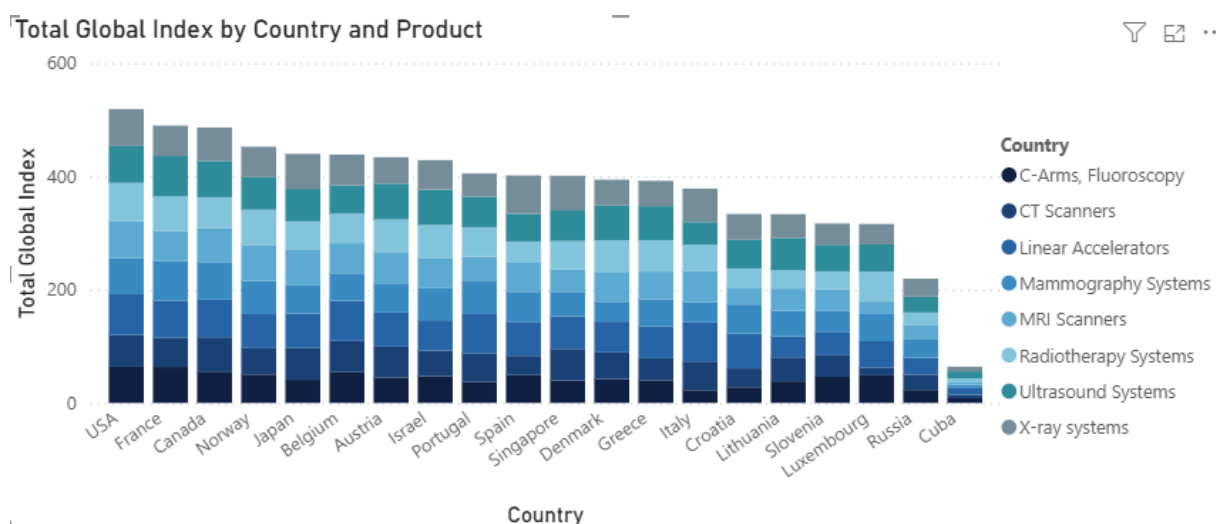


Fig. 11: Stacked Column Chart.

Historical and Forecast Sales Index

This visualization combines historical sales index values (2010–2024) with machine learning forecasts (2025–2027). A vertical reference line marks the transition between observed and predicted values, while the highlighted background visually distinguishes the forecasting period. This enables users to compare historical trends with projected market evolution in a single visualization.

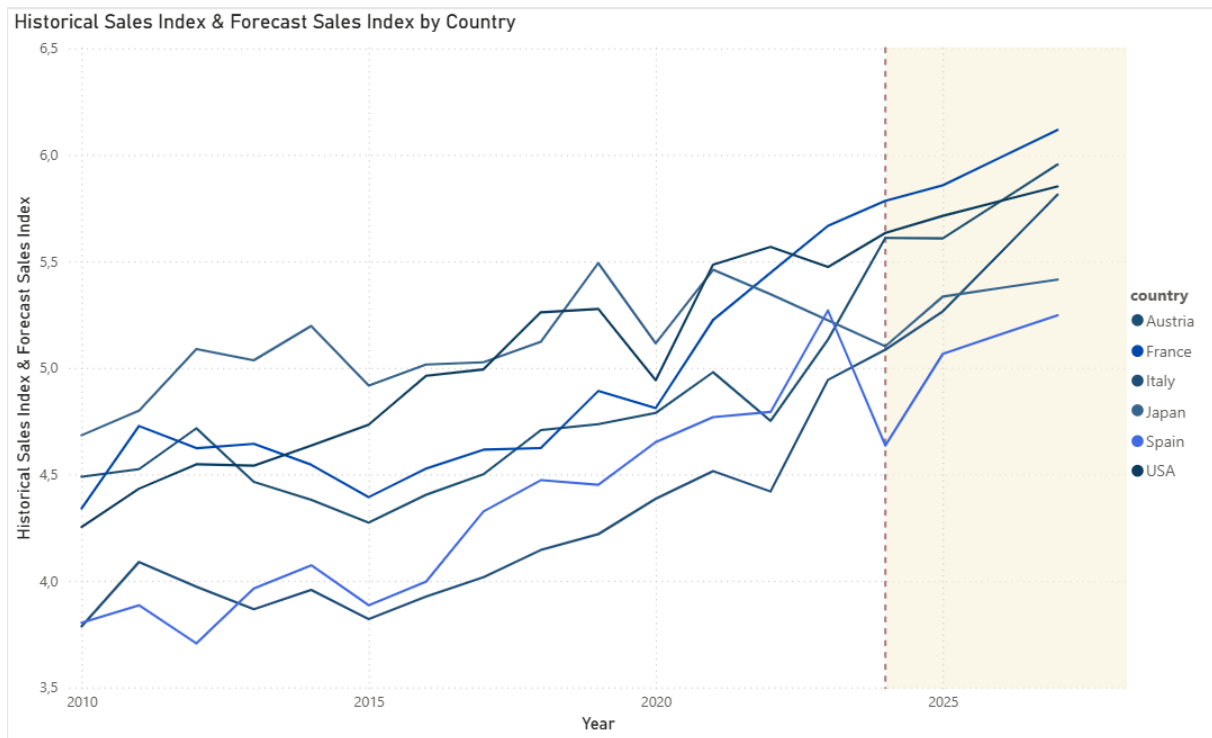


Fig. 12: Forecast by Country.

Key Performance Indicators (KPIs)

The Power BI dashboard includes several DAX measures designed to support business decision-making:

- **Total Annual Sales Index:** Aggregated annual sales potential across the selected data.
- **Total Global Sales Index:** Overall commercial attractiveness index used to rank countries and products.
- **Average Sales Index:** Average annual sales index for the selected data.
- **Top Country:** Country with the highest Global Sales Index.
- **Top Product:** Product with the highest Global Sales Index.

These KPIs provide a concise overview of market performance and allow users to quickly identify the most attractive countries and healthcare products.

VII. Results

The developed analytical pipeline successfully integrates data acquisition, preprocessing, market attractiveness assessment, predictive modeling, database management, and interactive visualization into a single Business Intelligence solution.

The generated results allow decision-makers to identify the countries and healthcare products with the highest commercial potential while also providing short-term sales forecasts based on historical trends.

Result 1: Country Ranking

The analysis shows that the countries with the highest potential for selling our products are the USA, Norway, France, Japan, and Austria.

Overall, the differences between these countries are relatively small. The same can be observed for the remaining countries included among the 18 most attractive markets for selling our products.

Initially, the ranking included the top 20 countries. However, the last two positions (Russia and Cuba) show considerably lower indicator values and were therefore excluded from the final ranking.

Result 2: Product Ranking

The two most demanded medical devices are linear accelerators and ultrasound systems. X-ray systems, mammography equipment, and radiotherapy devices are also highly demanded, with indicators close to those of the top-ranked products.

Overall, all eight medical devices show high market demand, meaning that the decrease in the indicator across the ranking is gradual.

Result 3: Sales Evolution

The overall sales evolution shows an upward trend. A significant decrease can be observed in 2020, the year in which the consequences of COVID-19 had a major impact on global markets. However, a clear recovery can already be seen in 2021.

Result 4: Market Overview

The geographical map allows us to quickly identify which countries have the highest potential and where sales efforts should be focused. This visualization helps identify the regions with the greatest business opportunities.

Result 5: Predictive Analysis

The forecast has been integrated into the Power BI dashboard, allowing users to compare historical and predicted values in the same visualization.

Holt's exponential smoothing was used to forecast future Annual Sales Index values for each country-product combination.

Predictive Model Evaluation

The forecasting model was evaluated using two standard regression metrics:

- **MAE (Mean Absolute Error)**, which measures the average prediction error.
- **RMSE (Root Mean Squared Error)**, which gives greater importance to large prediction errors.

Both metrics were calculated for every country-product combination and stored in the PostgreSQL database.

The obtained values indicate that Holt's Exponential Smoothing provides a satisfactory fit for short-term forecasting and can therefore be used to estimate future Annual Sales Index values.

VIII. Technologies Used

This project is based on the following technical components:

- Modular Python architecture
- Automated ETL process
- PostgreSQL relational database
- Interactive Power BI dashboard
- Predictive analytics
- Scalable software architecture

Scalability is one of the main strengths of this project. Two modular structures have been implemented; however, the architecture can easily be extended by incorporating additional analytical modules.

For example, a new module could be developed to include logistics indicators or other business variables. This new module would simply be integrated into the existing architecture, while the upper-level module would calculate the new global indicators. Consequently, the **Countries** and **Medical Devices** modules would remain unchanged and could continue to be reused without modification.

The following table summarizes the technologies used throughout the development of this project.

Technology	Purpose
Python	ETL, preprocessing and predictive analytics
Pandas	Data manipulation
NumPy	Numerical computations
Scikit-learn	Data preprocessing
Statsmodels	Holt exponential smoothing forecasting
PostgreSQL	Relational database
SQLAlchemy	Database connectivity
SQL	Data management
Power BI	Dashboard development
DAX	KPI calculations
Git	Version control

IX. Conclusions

This project demonstrates the complete development of a Business Intelligence solution for supporting strategic decision-making in the healthcare products sector.

The implemented pipeline integrates data acquisition, preprocessing, indicator calculation, predictive analytics, database management, and interactive visualization into a single automated workflow.

The modular Python architecture allows each analytical component to be developed, tested, and maintained independently. This design also facilitates the future incorporation of additional analytical modules without modifying the existing structure.

The integration of PostgreSQL and Power BI provides an efficient environment for storing, querying, and visualizing the generated information through interactive dashboards.

Finally, the incorporation of predictive analytics extends the descriptive analysis by providing short-term sales forecasts, allowing decision-makers to anticipate future market trends rather than relying exclusively on historical information.

Overall, this project combines data engineering, business intelligence, and predictive analytics within a scalable architecture that can be extended to support future business requirements.

X. Key Skills Demonstrated

During the development of this project, the following technical skills were applied:

- Python programming
- Data cleaning and preprocessing
- ETL pipeline development
- Modular software design
- Predictive analytics
- PostgreSQL database design
- SQL querying
- SQLAlchemy integration
- Power BI dashboard development
- DAX measures and KPIs
- Business Intelligence reporting

XI. CONTACT

Email: vtouza@yahoo.es

LinkedIn: www.linkedin.com/in/victor-touza-15143927

GitHub: <https://github.com/Victor-Touza/Healthcare-Market-Intelligence>